

Question	Answer
Whats the role of temperature here.	Temperature controls how random the model's word choices are. Low values like 0 or 0.2 make it pick the most likely words, so answers are focused and repeat well. Higher values like 0.7 or 1 let it pick less likely words, so answers are more varied and creative. Here 0.7 gives a natural, slightly varied explanation. For factual or extraction work, keep it near 0.
hosted server means - on HF or open AI ?	Hosted server means the model runs on someone else's machine and you just call it over the internet with an API key. That includes both OpenAI and Hugging Face Inference endpoints, and others like Groq or Together. The opposite is the first slide, where the model is downloaded and runs on your own laptop with transformers and torch.
I wanted to know is there any standard structure for creating the Prompts? Such as at which level we have to specify the input, output instructions in terms of where the model will do better inference when it needs the context of certain parts of instructions in the prompt?	There is no single universal prompt structure, but a useful pattern is: "Instruction -> Context/Input -> Constraints -> Output_Format" Put the main task first, then provide the relevant context, followed by rules such as "use only this context", and finally specify the expected output format. Clear constraints and explicit output requirements generally make the response more reliable and easier to parse.
In an API call, what is the purpose of the temperature parameter? How can I determine the appropriate temperature value to use for my specific use case?	Temperature controls randomness in the model's word choice. Low means safe and repeatable, high means varied and creative. Pick it by the task, then test. Say 0 to 0.3 for extraction, classification, SQL, maths and anything you want the same answer every time. Use 0.5 to 0.7 for chat, summaries and explanations. Use 0.8 to 1.0 for brainstorming and creative writing. The practical way is to run the same prompt five times at your chosen value. If you need consistency and the answers differ, lower it. If they feel dull or repetitive and you wanted variety, raise it.

Can you pls share the details of configuring integration with OpenAI. Including how to configure the credentials	Only if you are purchasing it. The following instructions apply - Create an account at platform.openai.com, add billing, then go to API keys and create a secret key. Copy it once, it is shown only that time. This is general - whenever you get the key. Store it as an environment variable named <code>OPENAI_API_KEY</code>. On Mac or Linux add <code>export OPENAI_API_KEY="your-key"</code> to your shell profile. On Windows use <code>setx OPENAI_API_KEY "your-key"</code>. For a project, put it in a <code>.env</code> file and add <code>.env</code> to <code>.gitignore</code>, then load it with <code>python-dotenv</code>.
What is “system” input in this context? Is the python code snippet showing the system input as well?	The system input is the instruction that sets the model's role, tone and rules for the whole conversation. It is not the user's question, it is the standing brief the model follows while answering. Yes, the code shows it. In the messages list the first entry with role system and content "You are a helpful assistant." is the system input, and the next entry with role user is the actual question.
In practical implementation In a production application, should we treat the system prompt as a version-controlled software configuration and test every change using a regression evaluation set? Since model responses are probabilistic, what evaluation approach would you recommend before deploying an optimized prompt?	Treat the system prompt as code. Keep it in git, give it a version number, and never edit it live in production. Every change goes through the same review and rollback path as a code change. For evaluation, build a fixed test set of 50 to 200 real inputs with expected outputs or a pass rule for each. Run the new prompt on all of them, at low temperature for stability, and run it three times per input so you can see variance, not just one lucky pass. Score on what matters: correctness, format validity, refusal rate, latency and token cost. Compare against the current prompt as the baseline and ship only if it wins overall and breaks nothing that used to pass. After that, roll out to a small share of live traffic first and watch the same metrics before full release.
We've seen the OpenAI API pattern <code>client.chat.completions.create()</code>. What other common standards or API patterns are used to interact with LLMs, such as the OpenAI Responses API, Anthropic Messages API, Google Gemini API, and OpenAI-compatible APIs? Is there an emerging industry standard for model API interoperability across different providers?	Yes. Common patterns include OpenAI's Responses API, Anthropic's Messages API, Google's Gemini API, and the older OpenAI Chat Completions pattern. Many providers also expose an OpenAI-compatible API, where the same <code>client.chat.completions.create()</code> interface can be used with a different <code>base_url</code>. However, there is not yet one formal universal standard for LLM APIs.

while sending input to model, how to reduce token cost for chain of thoughts/thinking token?	Strictly from personal experience, and ideas can differ - Do not ask a reasoning model to "think step by step" in the prompt. It already reasons internally, and that instruction makes it spend more thinking tokens for no gain. - Use the reasoning effort only for hard maths or code. Low or medium is enough for most tasks - Match the model to the task. Simple extraction, classification and formatting do not need a reasoning model at all, so use a cheaper standard model. Keep the reasoning model for the hard cases. - Keep the input tight. Trim long context, drop old chat, and ask for a short final answer format so the visible output is small.
can you please explain the difference between temerature and top-p as both are used to get creative control and diversity	Temperature is a global rescaling -- it doesn't remove any tokens from consideration, it just changes how much the probabilities differ from each other. Top-p works differently: it truncates the candidate pool of tokens. You pick a threshold p (e.g., 0.9), then the model sorts tokens by probability and keeps adding tokens starting from the most likely until their cumulative probability reaches p.
What is Chain of thought?	Chain-of-thought is the model's step-by-step reasoning process used to work through a problem before producing the final answer. Reasoning models are specifically trained to perform more of this internal reasoning, which can improve performance on complex, multi-step problems.
How is the LLM calculating the total distance travelled as 150 miles in total? Why is it missing the 2nd piece of information in the calculation?	The model likely calculated only the first part: $60 \times 2.5 = 150$ miles, and then generated the answer without checking the second part. The second part should add $40 \times 1.5 = 60$ miles, so the correct total is $150 + 60 = 210$ miles. This example shows how a standard model can jump to an answer from an incomplete reasoning pattern, while a reasoning model explicitly checks each step

who feeds the chain of thought to model?	Nobody feeds it. The model produces the chain of thought itself. You send only the question, and the model generates its reasoning tokens first, then the final answer, all in the same generation.
who feeds the chain of thought to model?	The difference from earlier prompting is where it comes from. In chain-of-thought prompting on a standard model, you trigger it by writing "think step by step" or by showing worked examples. In a reasoning model it is baked in by training, so it happens on its own even if you say nothing.
For an enterprise application, how should we decide when to route a request to a reasoning model instead of a standard model?	- Route by task difficulty, not by importance. - When the answer needs several dependent steps that can go wrong: multi-step maths, financial or eligibility calculations, code debugging, comparing rules across documents, or planning where one wrong step breaks everything.
Please discuss in a bit detail, how these models are trained, how the data for training is fetched etc.	Usually models are trained in stages: - Pretraining : here the models learn general language, facts. The data comes from web crawls, and large amount of data from the internet. Data is preprocessed like deduplication, PII scrubbing. Following this, the models are pretrained using unsupervised / self-supervised training. Why self-supervised? That's cause you will have the next token in your data to guide the training. - Fine-tuning : Once you have the pretrained models, you would like to make a specialised model for your task. That's where you have a specialised set. of data like health data and then train on it using supervised / instruction tuned method. - RLHF / Reinforcement learning using human feedback : This stage shapes behaviour using human responses as feedback. You must have seen on chatgpt, you are asked to choose which answer you find good. That choice is chosen as the preferred answer, and the models are further trained using reinforcement learning.
Please discuss in a bit detail, how these models are trained, how the data for training is fetched etc.	Training recipes for closed source models are not documented (hence it's closed source). For open-weight models like gemma, qwen, the training methods are published in their papers. High quality data ensures a well-trained model.
what do you mean by open weights model?	Open weights model are models whose parameters/weights are available for download such that people can download them locally.

but the o/p is same in both of the cases in context of an end user , so how its help	Look at the slide example again. The outputs are not the same. The standard model says 150 miles, which is wrong, it missed the second leg. The reasoning model says 210 miles, which is correct. The end user sees only the final line in both cases, so the difference is invisible in style but very visible in accuracy. The extra thinking is not for the user to read, it is what catches the mistake before the answer is shown.
The standard model returns 150 miles in total which is incorrect. Lets say some such computation was deeply embedded inside a larger top level task. How will the user know such an error occurred which triggered a chain of errors subsequently due to logical dependencies? Any recommended mitigation strategies? Is this still a concern with current frontier models? Is a reasoning model the same as setting the thinking effort to high on non-explicit reasoning models?	Yes, this is still a concern. Even frontier reasoning models can make intermediate errors, and a wrong early step can propagate through all dependent steps. Current reasoning models are substantially better at detecting and correcting such errors, but they are not guaranteed to be correct. A key mitigation is to verify important intermediate results, rather than trusting only the final answer.
As a project managent professional, how will i make sure the budget of the tokens. Adding reasoning models the tokens assessments are beyond control	True. One needs to experiment a bit to ensure what will be the number of tokens suitable for your task. You can check what would be the average tokens per call -- that will be input + output + reasoning tokens. You can cap `max_tokens` per request and check how many tokens would benefit your task. If let's say your task is some form of classification, perhaps you would not need much reasoning tokens. If it's open ended, you may need a bit of reasoning tokens more. You can also exploit caching -- reuse system prompts and static context via. prompt caching. Repeated context workloads can cut cost by a large amount.
What happens technically when we click answer now button when it is in middle of thinking in Gemini/ ChatGPT?	When you click Answer now, the system interrupts the extended reasoning phase and asks the model to produce its final response immediately. Technically, this reduces the remaining reasoning_compute or thinking_tokens allocated to the request, so the answer arrives faster but may undergo fewer intermediate checks. It does not reveal the partially generated reasoning, the model simply transitions to final-answer generation.
how does that download help?	Could you point me to which download you mean, the model download in the transformers slide or something the instructor just showed? If it is the transformers slide, downloading the model means the weights sit on your own machine, so the model runs locally instead of calling an API. That gives you no per-call cost, no data leaving your laptop, and it works offline. But you need enough RAM or GPU, and small local models are weaker than the hosted ones.

Are there Models which are still not using reasoning/chain of thought? What are the use cases for such Models.	The standard chat models like GPT-4o-mini, Gemma, Llama and Mistral answer directly without an internal thinking phase. When the answer is a lookup or a transformation rather than a multi-step deduction. Classification, sentiment, entity extraction, summarising, translation, rewriting, formatting into JSON, drafting emails, chat replies. We will be seeing more use cases as we progress in the course.
what is exactly meant by rewarding? how do the open ai know if the ans is correct or wrong in real time scenario?	“Rewarding” means giving the model a numerical feedback signal indicating how good its generated answer was. The model is then updated so that responses producing higher rewards become more likely.
so generating chain of thought minimizes error?	Generally, yes, but not guaranteed. Chain-of-thought gives the model more intermediate reasoning steps, which can help catch mistakes and solve multi-step problems. However, it can also make mistakes in those intermediate steps and propagate them.
How HuggingFace can be executed locally? Eventually the request goes to model hosted on a remote server?	No, once the model is on your machine nothing goes to a remote server for the actual answer. The first time you run <code>from_pretrained</code>, the transformers library downloads the model weights from the Hugging Face hub and saves them in a local cache folder on your disk. That is Approach 1 on the slide. Approach 2 is the opposite, there the model stays on the provider's server and only your text travels back and forth. Approach 3 with Ollama is local like Approach 1, it just wraps the local model behind a small API on your own machine so your code can call it like a normal endpoint.
Are there any models which use chain of thoughts if required and for simpler tasks it doesn't? Is this combination possible?	Yes. These are hybrid reasoning models, like Claude, Gemini and Qwen3. They think only when the question needs it and answer directly otherwise. Most also let you set a thinking budget or reasoning effort to control it.
How can I host LLM on my laptop or my cloud account?	On a laptop, easiest is Ollama. Install it, run <code>ollama pull llama3</code>, then <code>ollama run llama3</code>, and you get a local API on port 11434. Or use transformers directly as in Approach 1. On cloud, take a GPU VM and serve the model with vLLM. It gives you an OpenAI-compatible endpoint your code can call.
Can you give me some examples on SLMs and Frontier models and how do they differ?	SLMs are smaller, efficient models such as Gemma 3 4B or Phi-4 Mini. Frontier models are much larger, state-of-the-art models such as GPT-5, Claude, or Gemini. SLMs prioritize efficiency, frontier models prioritize maximum capability.

How LLM always responds in positive collaborative tone? which parameter is responsible for this?	There isn't a single parameter responsible for it. The system/developer prompt and post-training (e.g: RLHF) teach the model to maintain a collaborative tone.
third party can be organization server/machine too? and if yes then how it processing flow on different server?	Yes. Your company can host the open model on its own GPU server, and that is the common enterprise setup. Flow is the same as any API. Your app sends the prompt over the internal network to that server, vLLM or Ollama runs the model there and returns the text. Nothing leaves your network, so data stays in house.
what are the system requirement to use model locally?	Rough estimates only from experience - RAM or VRAM needed is about 2 GB per billion parameters in float16, half that if quantised to 4-bit. So a 2B model runs on 8 GB RAM, 7B or 8B needs 16 GB comfortably, 13B wants 32 GB.
approach 1 vs 3 example please	Approach 1, model runs inside your Python process: <pre>from transformers import AutoModelForCausalLM model = AutoModelForCausalLM.from_pretrained("google/gemma-2-2b-it") outputs = model.generate(**inputs)</pre> Approach 3, Ollama runs the model as a service, you call it: <pre>requests.post("http://localhost:11434/api/generate", json={"model":"gemma2","prompt":"Hi"})</pre> Both are local. Difference is 1 loads weights in your code, 3 talks to a local server.
when honourable professor mentioned that for Approach 1 we can know technicalities of model - what does that mean and how to get that parameters	It means you hold the actual model object, so you can inspect it. <code>print(model)</code> shows every layer, <code>model.config</code> gives hidden size, layers, heads and vocab size, and <code>sum(p.numel() for p in model.parameters())</code> gives the parameter count.
Can we run SLM on edge devices? like mobile with low config. if Yes, can we discuss on this topic.	Yes. Small models of 1B to 3B, quantised to 4-bit, run on modern phones. Examples are Gemma 2B, Phi-3 mini, Qwen 2.5 1.5B and Llama 3.2 1B. As for discussion on the topic, we would have a hand on session. If it is not covered as per your expectations, you can reach out to the team.

Can we use a image clasification model locally when we dont have high configuration laptop? if yes then how?	Yes. You can use a small/pre-trained image classification model locally even on a low-spec laptop. You can use lightweight models such as MobileNet or EfficientNet-Lite. You download the pre-trained model, load it locally, and run inference on your images.
what is tokenizer in this context?	It's a function derived from the AutoTokenizer class that will be dividing the input text into different tokens or units that will be processed by the models to predict next tokens or answer the question. E.g. for "what is quantum computing?" -- "what", "is", "token", "computing", "?" will be the tokens.
When using hugging face api to load model locally, it will run on same server/laptop. Is it required high system config like GPU? or whats the advantage or loading using ollama.	Yes, same machine either way, so the hardware need is the same. A GPU is not mandatory, CPU works but is slow. Roughly 2 GB memory per billion parameters, half if quantised. Ollama's advantage is convenience. It ships quantised models by default so they need less memory.
is the request/response path generic for any LLM callouts like we have client.chat.completions.create()? Or is it configurable?	Mostly generic now. The OpenAI chat completions format has become the de facto standard, so HF router, Ollama, vLLM, Groq and Together all accept the same messages, model, temperature shape. You just change the base_url and key. Anthropic and Gemini have their own native formats, but both also offer OpenAI-compatible endpoints.
How we get to know what to pass in model key.	You copy it from the provider. On Hugging Face it is the repo name on the model page, like Qwen/Qwen3-32B, plus the provider suffix such as :nscale as in the slide. For OpenAI it is in their models docs, gpt-4o-mini. For Ollama it is whatever you pulled, seen with ollama list.
Are there any tools the explanation above is not convincing for me as AI project manager at consulting I need to provide an estimate which should not deviate much.	Token estimation itself is not an easy task I would say. It will be an iterative approach from task requirements to continuous monitoring. I think rather than giving the clients "expect 500M tokens/month", it would be good to get a rough idea first by running your task on a pilot level and then give different tiers like base case : 500M tokens/month, planning case : 650M tokens/month, stress case : 800M tokens/month. Regarding tools, you would have to have a system that records every LLM call - what's the application, use case, model, input tokens, output tokens, tool calls, latency. i have not tried but some tools like Langfuse, MLflow can help you observe the flow of your pipeline that can help gauge the cost or tokens being spent.

Thanks Soumi, Can you pls point me to some reference material, to go in depth	Sure. Attaching some papers that you can go over to understand better: https://proceedings.neurips.cc/paper_files/paper/2022/file/b1efde53be364a73914f58805a001731-Paper-Conference.pdf https://proceedings.neurips.cc/paper/2017/file/d5e2c0adad503c91f91df240d0cd4e49-Paper.pdf Qwen 3 technical report - https://arxiv.org/pdf/2505.09388 Gemma 3 technical report - https://arxiv.org/pdf/2503.19786
Should same model be used for tokenizer and prompt processing ?	Yes and always. Every model has its own vocabulary and token IDs. Using a different tokenizer produces IDs the model never learnt, so you get garbage output. That is why the slide passes the same model_id to both AutoTokenizer and AutoModelForCausalLM.
How can we assure that hugging face is secure	Pick official org repos like google, meta-llama, Qwen, check the licence and downloads. Anyone can upload, so treat unknown repos with care. Prefer models saved in safetensors format, which cannot execute code.
what is nscale	nscale is one of the inference providers on the Hugging Face router, a company that hosts open models on its GPUs. The suffix in "Qwen/Qwen3-32B:nscale" tells HF which provider should serve that model.
If we send the request through API_URL = "https://router.huggingface.co/v1/chat/completions" does it not cause the issue of data security?	Since the request goes to Hugging Face's servers, the data in the prompt is leaving your machine, so there is always a potential data-privacy concern.
If we send the request through API_URL = "https://router.huggingface.co/v1/chat/completions" does it not cause the issue of data security?	If you need stronger privacy, you can use a model locally (e.g. with Ollama or Transformers).
what is the difference between Local hugging face vs Ollama	Both run the model on your own machine, so privacy and hardware needs are the same. The difference is how you use it. Transformers loads the model inside your Python code. Ollama is easier and lighter, but you cannot look inside the model.

is the 3rd approach Ollama requires more hardware storage to run it locally?	Not really. Both Ollama and the Transformers approach store the full model locally, so storage depends on which model you pick, not the framework itself. A 7-billion-parameter model takes roughly the same disk space whether you run it with Ollama or with Transformers. The difference is that Ollama bundles its own runtime, so you don't need PyTorch installed separately. Storage is determined by model size, not the tool you choose to run it.
My understanding is - The prompt is hardcoded here only to demonstrate how the model works. In production, the application normally constructs prompts dynamically from user input, business context, retrieved information .	Exactly right. The hardcoded string is just for the demo. In production you build the prompt at runtime from a template, filling in the user's question, session or business context, and any retrieved chunks from your knowledge base. The system message usually stays fixed and version controlled, while the user message is assembled per request.
Also a bit basic question, is the system resource req for training the models, similar to running requirements or are they drastically different, if different then which requirement is higher.	Good question! Training requires far more resources than running (inference). Training computes and stores model gradients for every parameter across the dataset to update. A rough estimate for a 1B model, roughly you would need 20-25 GB memory (including weights, model gradients, optimiser states, activations). Inference just does a forward pass -- you already have the fixed weights, and you are not updating anything -- so you don't need anything for gradients, optimiser states. For inference, a very important thing you need for these LLMs is KV cache. Depending on your input, it will be storing the K (key) and V (values) of input tokens in the cache so that the model can predict the next token faster.
What is ollama really? What does it mean when we say it runs the model? Is it minimal inference engine using the local computers resources?	Yes, that is exactly right. Ollama is a lightweight inference server built on llama.cpp. It bundles its own runtime, so no torch needed. Running the model means it loads the quantised weights into your RAM or GPU memory, keeps them warm, and does the token-by-token maths using your CPU, GPU or Apple Silicon. It then exposes that as a local REST API on port 11434.
between all 3 which one is more preferable in production	For real production you would use vLLM on your own GPU server, which handles batching, concurrency and throughput properly. Of the three on the slide, Ollama is the closest for a small internal service, transformers only if you need to fine-tune or inspect the model, and HF free inference is for prototypes only because of the rate limits.

does ollama require license? and why orgs are not actively using it? any trade off? since it may solve data sending to other provider issue. correct me if wrong	Ollama is open-source and completely free, no license needed. You're right that it keeps data private locally, which solves the privacy concern. But organizations avoid it because smaller models running locally are less capable than cloud models like GPT-4, and running them demands expensive hardware with good GPUs. There's also operational overhead maintaining the infrastructure. For many orgs, paying per-token to a cloud provider is cheaper and simpler than buying and maintaining powerful hardware in-house.
In scenarios where prompt instructions are huge leading to higher number of input tokens. Is it possible to compress the prompt and then send to model for processing? Would such compaction result in tangible cost savings ?	Yes, this is called prompt compression, and tools like LLMingua and LLMingua-2 do it by dropping low-information tokens. Compression itself costs a small model run, and accuracy can drop if you cut too hard. So savings is there but you have to verify whether it means much on your data.
For application dealing with Patient Data, what approach is used generally ?	Self-hosted, almost always. An open model like Llama or Mistral served with vLLM on your own or your cloud VPC GPU, so patient data never leaves your controlled environment.
How can approach 3 run 7B model but not approach 1?	Approach 1 can run 7B too, it is just heavier by default. Transformers loads full or half precision weights, so a 7B model needs roughly 14 GB. Ollama ships 4-bit quantised versions, so the same 7B fits in about 4 to 5 GB.
What is proprietary model?	A proprietary model is owned and controlled by a company like OpenAI or Anthropic. They don't release the model weights or internal details publicly. You can only use it through their API, paying per request. In contrast, open-source models on HuggingFace are freely available with all code visible so you can inspect and modify them. Proprietary models are usually more capable because companies invest heavily in training, but you lose transparency and must send data to their servers.
When we are downloading Ollama means we are downloading the trained Model only right if we are hosting Locally from our end we will be bearing the Computation cost during Inference but these models can we download or install freely?	Yes. The model itself can often be downloaded for free, depending on its license. But you bear the hardware/computation cost during inference (CPU/GPU, RAM, electricity).

why arent the organizations restricting the usage of hugging face models?	Some organizations actually do restrict them for security and compliance reasons. But many don't because HuggingFace models come from trusted researchers and the community audits them. Plus running locally avoids sending sensitive data to external servers, which actually improves security. For non-critical tasks, open-source models are cost-effective and good enough. Large enterprises with strict governance might stick to proprietary APIs for control and liability, but smaller companies benefit from the freedom and savings HuggingFace offers.
Does the model download size on hugging face defines how much GPU ram is needed ? Also on MAC with 48GB RAM as they publish its unified RAM (CPU+GPU) when i loaded Qwen 3.8 27Billion parameter and prompted it to do a programming taks it took 1000sec with 37K tokens being used . Should it take that long or the model swtiched from using GPU to CPU ?	Yes, download size is a good proxy. The weights file size is roughly what must sit in memory, plus 1 to 3 GB extra for KV cache and activations. On 48 GB unified memory a 27B model in 4-bit is around 16 to 18 GB, so it fits and should stay on GPU. Your number is probably not a CPU fallback. 37K tokens is a very long context, and attention cost grows sharply with context length, so the KV cache alone can add many GB and slow decoding a lot. Check it. Run with a short prompt and compare tokens per second. Watch memory pressure in Activity Monitor. If short prompts are fast and only the long one crawls, it is context length, not device switching.
tokenization and decoding the tokens varies is model specific?	Exactly right. Each model has its own tokenizer that defines how text splits into tokens and maps them to integer IDs. A token ID might represent different text across different models. That's why the code always loads the matching tokenizer with the model using <code>'AutoTokenizer.from_pretrained(model_name)'</code>. Using the wrong tokenizer with a model breaks everything because the model's weights were trained on that specific tokenizer's vocabulary. So tokenization and decoding are tightly coupled to each model.
in closed-domain model,what is the use of training?is it required?	No extra training is needed. You still need a pre-trained model, but only for its language ability, understanding the question, reading the passage, and writing a clean answer. The facts come from the passage you supply, not from training. That is why RAG works with an off-the-shelf model. Fine-tuning is optional and used only to fix style, format or domain vocabulary, not to add knowledge.

The company chatbot is more of a RAG system or a complete LLM setup?	A company chatbot is often a RAG system built on top of an LLM, rather than being a separate type of LLM. The LLM provides the language capability, while RAG retrieves relevant company documents and gives them to the LLM as context.
How can we be sure the model did not use info learned during training to answer a closed domain QA?	You cannot be 100% sure from the model's answer alone. For closed domain QA, you need an experimental control. You can create questions whose answers exist only in your private corpus, and that will unlikely be in your pretraining. If the model answers without the corpus, that's an evidence of memorization.
Search Engines should be closed domain...are we saying search engines does has answers from prior knowledge	Closed domain and prior knowledge are two separate axes - not opposite of each other. Closed domain is when it is restricted to a specific area. Prior knowledge means closed-book i.e. where the model answers purely from what is has learned during training. So closed domain may not be in the model's prior knowledge. Now coming to your question - depends on what layer you are looking at. A traditional SE doesn't answer at all. It retrieves and ranks documents. There's no generation, so prior knowledge doesn't apply, it's pure retrieval. An AI powered search like Google AI overview, or Perplexity retrieves documents (using their web search tools) and then use an LLM to generate an answer. Here the key thing is " the LLM's generation step is not purely closed to the retrieved documents, but the underlying model has all its prior knowledge, it blends in that knowledge, and alongside the retrieved content from the web search tools, it answers.
What is SQuAD and MMLU	SQuAD (Stanford Question Answering Dataset) is a closed-domain QA dataset where the model gets a passage and must answer a question using only that passage. MMLU (Massive Multitask Language Understanding) is an open-domain benchmark containing multiple-choice questions across many subjects, testing general knowledge and reasoning.
Thanks @Amit. Kindly let me know vLLM refers here is vision LLM ?	No, different thing. vLLM is an open-source inference server for running LLMs efficiently on GPUs, using paged attention and batching for high throughput. That is what I meant for production serving. Vision language models are usually written VLM, without the small v.

How about code generation if customer wants to have the reference only from internal repo and restricting model to go outside of customer network to derive it from public repo. Will it be closed domain	Yes. If the model is restricted to the customer's internal repository/context and must derive the code only from that information, it is a closed-domain code-generation task.
For the load_dataset, is there a specific format of dataset? Lets say I have some enterprise specific data over which I need to run the AI, should i convert it to some specific format first or could it be some arbitrarily large key value pair file?	load_dataset is flexible -- CSV, JSON/JSONL, parquet, text, pandas, python dictionary etc. all work, and yes, an arbitrary key-value JSON/JSONL file is also fine ; you don't need a strict predefined schema.
For the load_dataset, is there a specific format of dataset? Lets say I have some enterprise specific data over which I need to run the AI, should i convert it to some specific format first or could it be some arbitrarily large key value pair file?	You can check https://huggingface.co/docs/datasets/v4.8.4/en/package_reference/loading_methods#datasets.load_dataset:~:text=Load%20a%20dataset%20builder%3A for the detailed list as well.
Is there any practical examples of evaluating prompts in such a complicated way when i can have an AI assistant do all of this is give my success criteria and constraints? ultimately this is going to be a one time exercise before setting up the work flow	Yes you can have an assistant generate the harness and even the prompt variants. But the run itself still has to happen, because only your data tells you which variant wins, and results differ by model as this slide shows. It is also not one time. It reruns every time you change the model, the prompt, or the provider version. That is why it is worth having as a script, not a manual exercise.
what is fallback in option B of original question	fallback is the backup path used when constrained generation is not available. You can think of it as an Exception block in normal programming. E.g. You're building a system where an LLM extracts structured invoice data as JSON: {"vendor": str, "amount": float, "date": str}. You use a model/provider that supports strict JSON schema enforcement: {"type": "json_schema", "strict": true}. The model is forced to only emit tokens that produce valid JSON matching your schema. Fallback happens for e.g. when say you switch to a smaller/cheaper model and it doesn't support struct schema or constrained decoding. So you have a fallback where you prompt it normally with "Respond only in this JSON format:...", if parsing fails, you can retry once with another instruction like "Your last response wasn't valid, fix it and return ONLY JSON".

Is ARC_Easy dataset only for evaluation of prompts of multiple choice question problem or can it be used for other problems?	ARC-Easy is purely multiple choice, grade-school science questions with four options. It is used here only because scoring is trivial, the answer is one letter, so you can measure prompt quality automatically. For other tasks, and I have used depended on use cases - SQuAD for QA, CNN, XSum for summarization, HumanEval for code, GSM8K for maths reasoning. People have also tried building a small set from your own real data and it was a better approach in most cases.
Is there a chance that the model efficiency might vary with increasing number of data set maybe from 2000 to 10000?	The evaluation set size or something in the notebook? Answering for both. If you mean the eval set, the model itself does not change. What changes is your confidence in the measured score. On 20 questions the numbers are very noisy, one question is five percent. At 2000 or 10000 the estimate becomes stable, so gaps between prompts you can trust. If you meant context length, that is different. Longer inputs do slow generation and can hurt accuracy, since models lose track of details in the middle of long context.
How can a smaller model like Qwen3-1.5B outperform a larger 3.8B model, and what factors besides parameter count influence model performance?	A smaller model like Qwen3-1.5B can outperform a 3.8B model if its training, prompt alignment, and reasoning behaviour are better suited to the task. In the example, Qwen3 performs very well with constrained prompts but drops sharply with open-ended reasoning, showing that prompt design matters a lot. Besides parameter count, performance depends on training/post-training, task fit, reasoning settings, decoding strategy, and how clearly the input/output constraints are specified.
there must be a way to control/optimize thinking token limit. right? and what is best combination of accuracy vs parse api ?	Yes. Most reasoning models expose it. OpenAI has reasoning_effort with low, medium, high. Accuracy is what you optimise. Parse rate just tells you the answer came out in a readable format. A prompt with high parse rate but low accuracy, like B on Qwen3-1.5B, is worse than useless because it fails confidently. So pick the highest accuracy, and treat a parse rate below about 95 percent as a prompt formatting problem to fix rather than a trade-off to accept.

IS tokenizer also a Model or just a Program if program how does it know how to chunk the input that chunks Input into Tokens and should we use same Tokenizer for a Model?	A tokenizer is usually not a deep learning model, it is a preprocessing program with a fixed vocabulary and tokenisation rules that convert text into token IDs. The exact chunking method depends on how that tokenizer was built, for example using subword units, but the key point is that those token IDs must match what the LLM was trained on.
The use case i can think is , if i want to build a Training portal for employees 1. AI generates or recommends the learning content. 2. Normal programming conducts and scores the assessment.	That split is right, and it is how most production systems are built. Use the model for the generative part, content, explanations, question drafting, and keep scoring in deterministic code so results are consistent and auditable.
Had a question on these llm's like gemini , chatgpt , copilot - i understand that there are llm versions pretrained on certain parameters at some cut-off date and these are then exposed to us for prompt answering , now my question is that since these llm are able to do search in world wide web search also - so the answers with newer version of models which are better fine tuned - for reaserach type prompt what will be accuracy and hallucination in newer llm or the same old llm	Good questions! Web search access and beter fine-tuning are two separate improvements that affect accuracy and hallucination differently. Fine-tuned model can make the models prone to a specific task rather than open ended questions. For research work, newer model + search will be more accurate and less prone to hallucination than anolder model + search. However one thing to note is that the main gain comes from better handling of the retrieved information from the web search tools rather than the models "knowing mode".
By theory i understand LLm use matrix multiplication to compare the input token against its weights.Please correct me if i am wrong. But I am confused how for Billions and Trillions of parameters it does it in microseconds	Your understanding is right, it is matrix multiplication all the way through. The speed comes from parallelism. A GPU has tens of thousands of small cores doing multiply-add at the same time, and matrix multiplication is the ideal shape for that, every output cell is independent. A modern GPU does hundreds of trillions of these operations per second.
By theory i understand LLm use matrix multiplication to compare the input token against its weights.Please correct me if i am wrong. But I am confused how for Billions and Trillions of parameters it does it in microseconds	A 7B model needs roughly 14 billion operations per token. At 300 trillion per second that is well under a millisecond of pure compute. In practice the real bottleneck is not maths at all, it is memory bandwidth, moving those 14 GB of weights from memory to the cores for every single token. That is why a bigger model is slower mainly because it is bigger, not because it thinks harder.

if we dont have benchmark datasets for evaluation , like in the case of stock Forecasting , can we use backtesting or future testing(like making predictions and then validating those on future dates) or is there any other way ?	Yes, both are valid and standard. Coming from my experience - Backtesting is the usual first step, but it must be walk-forward. Train or prompt using only data available up to time t, predict t plus one, then roll forward. Never let future data leak into the input, that is what makes most backtests look good and fail live. Forward testing, paper trading on live data, is the honest check. Slower but no leakage possible, so run it before any money moves. For the LLM part specifically, remember the model may have seen the historical period in training, so a backtest on old data can be contaminated. Prefer recent out-of-sample windows and forward testing there.
How do we set any LLM like CLaude at enterprise level to scale it to 10000 + employees	For general use, take the enterprise plan and connect it to your identity provider with SSO and SCIM, plus data retention and admin controls. For apps your teams build, put an internal gateway in front of the provider. It holds the keys, enforces per-team budgets and rate limits, logs everything for audit, and lets you swap models without code changes.
The question here how to minimize the tokens	Some ideas are as follows: You can remove redundant instructions that you would also feel is unnecessary ; you can periodically compress old conversation turns into short summary and send as context ; you can cache thestatic prompts so that they are not re-processed ; you can cap output with max_tokens that avoids unneccsarily long generations ; you can sue structured output formats (you'll be taught next on this), batch related queries rather than repeating shared context.
How can we build a repository-aware code-generation LLMs ,that uses our existing source code, coding standards and architectural patterns as grounding context? Rather than using cursor ,Claud etc.	Build it as RAG over the repo. Index the code by function or class using an AST-aware splitter, embed with a code embedding model, and store in a vector DB. On each request retrieve the relevant files plus your standards doc, and put them in the prompt.

What is a technical aspect behind Context window? Why do we define context window for the frontier models? Is it based on the servers load capacity as these are multitenant i believe?	The context window is the maximum number of past tokens, including the prompt, conversation history, and any retrieved content, that a model can attend to when predicting its next token, and this limit is set primarily during training, as in what the model actually saw during training, so context window is a fixed, published property of each model (e.g., 128K, 200K, 1M tokens) rather than something configured at inference time. We don't define context window for the frontier models, we define the number of tokens (max_tokens) in which that the model should answer. This is distinct from max_tokens, which is a separate, per-request parameter you as the developer set to cap how many tokens the model is allowed to generate in its response. Context window governs how much the model can see, while max_tokens governs how much it's allowed to say.
When we send a prompt to an AI system for a major code change, the model sometimes takes longer to process ,makes certain changes and the request times out. What are the best practices or strategies to handle such timeouts — for example, should we break the request into smaller chunks, use streaming responses, or implement retry mechanism	This specific timeout-handling topic is not covered directly in the material; in practice, the best strategy is to break a large code change into smaller, independently verifiable steps so each request finishes faster and failures are easier to recover from. Streaming helps avoid client-side idle timeouts and provides partial output, but it does not reduce the model's actual computation time. Use retries with exponential backoff only for transient API or network failures, and make operations idempotent so retries do not duplicate changes. For long tasks, save intermediate state and resume from the last successful step instead of restarting the entire workflow.
Lets say i have a calculation to be done or i am asking some framework to be generated based on reasoning and the prompt. Now i am tracking the framework execution how do i ask the model not to compress or remove the framework details when the context window size is going to fill. How to make prompts in such a case ?	Great question! Two separate problems: - Model summarizing on its own (fixable via prompting): You can explicitly instruct it not to compress e.g., "Reproduce the full framework verbatim; never summarize or drop prior sections, even for brevity." Wrapping the framework in tags (<framework>...</framework>) also helps. - Hard context window limit (not fixable via prompting): Once content falls outside the window, the model literally can't see it. No instruction fixes this. You can keep the framework as an external oracle or ground truth (a file/doc), and re-paste the full current version at the start of each new prompt instead of relying on the model to remember it from earlier turns.

Is there any way to persistently store my framework generated from Reasoning Model?	Yes, but store the output, not the reasoning. Once the model produces the framework, save it as a file, a document, or a Pydantic-validated JSON record in your own database or repo. Then reuse it as context in later prompts instead of regenerating. The hidden thinking trace itself is not returned by most providers and is not meant to be reused.
One problem I run into frequently is this, I develop a code module using Opus 4.6 High thinking. It works fine. The regular tests look good. However, when I run this through a thorough code review prompt on Opus 5, it brings out some edge case scenarios that have not been handled and could lead to day0 production issues. I provide this feedback to 4.6 and it does an audit and it agrees to the findings. Practically, what could the issue here be? My prompting style, model selection for tasks or anything else? Just curious to know, frontier models from the same companies don't seem to agree on code artifacts generated by themselves. What techniques are likely to reduce these uncertainties for code that needs to get deployed in production on critical devices? This is ofcourse simplified. There are processes in place to ensure thorough testing etc. My query is will this doubt not arise in anyone's mind that it may always be safer to use the latest models quite simply because of their deeper intellect?	Nothing wrong with your prompting. Generating and reviewing are different tasks. While writing, the model is optimising for a working solution and commits to a path early. In review it starts from finished code and hunts for failure, which is a much easier frame for spotting edge cases. A human writing then reviewing their own code shows the same gap. The disagreement across versions is also expected. Newer models catch more, and no model catches everything, so agreement is not the goal. Practical fixes are process, not model choice. Always separate the reviewer from the author, ideally a different model and a fresh context with no memory of the generation. Ask explicitly for edge cases, concurrency, failure modes, boundary values. Have the model write adversarial tests before you accept the code, and keep the human review and static analysis in the loop.
One problem I run into frequently is this, I develop a code module using Opus 4.6 High thinking. It works fine. The regular tests look good. However, when I run this through a thorough code review prompt on Opus 5, it brings out some edge case scenarios that have not been handled and could lead to day0 production issues. I provide this feedback to 4.6 and it does an audit and it agrees to the findings. Practically, what could the issue here be? My prompting style, model selection for tasks or anything else? Just curious to know, frontier models from the same companies don't seem to agree on code artifacts generated by themselves. What techniques are likely to reduce these uncertainties for code that needs to get deployed in production on critical devices? This is ofcourse simplified. There are processes in place to ensure thorough testing etc. My query is will this doubt not arise in anyone's mind that it may always be safer to use the latest models quite simply because of their deeper intellect?	On the last point, yes, newer usually reviews better, so use the strongest available model as the reviewer. But do not treat it as a guarantee. On critical devices the safety comes from tests and review layers, not from any single model's judgement.

Hi Soumi sir so if we need to objectively answer this question - newer better fine tuned model on deep reasearch - will be trained on any a-z web research tasks more accurately than prev version of llm so newer version of model will always be better to answer -lets say by 10-15% better	Hi Deepak, Yes, newer fine-tuned model will be better. Making it accessible to web search will be also helpful.
sir notebook link?	link to hand-on notebook: https://colab.research.google.com/drive/1KIUMnstBxwW9wdl4ml0lxr0detG-jnAl#scrollTo=TtkWRWZTWOQw
I do not see the hands on file available in the resources, how can I get that?	https://colab.research.google.com/drive/1KIUMnstBxwW9wdl4ml0lxr0detG-jnAl#scrollTo=TtkWRWZTWOQw
where can i find this notebook ?	https://colab.research.google.com/drive/1KIUMnstBxwW9wdl4ml0lxr0detG-jnAl#scrollTo=TtkWRWZTWOQw
This hands on code is not there in resource tab	https://colab.research.google.com/drive/1KIUMnstBxwW9wdl4ml0lxr0detG-jnAl#scrollTo=TtkWRWZTWOQw
Where can I get the notebook which is explained right now?	https://colab.research.google.com/drive/1KIUMnstBxwW9wdl4ml0lxr0detG-jnAl#scrollTo=TtkWRWZTWOQw
For option B, the model is hosted via HF and via option C, its remotely hosted but by another 3rd party such as upgrad in our case?	Yes. Option B goes through the Hugging Face router, which forwards to a provider like featherless-ai. Option C is UpGrad's own vLLM server at that IP, serving Qwen2.5-7B-Instruct-AWQ.
where is the model weight loaded? In google collab env?	Same OpenAI client in both cases, only base_url and key change.
does this dtype represent the datatype of weights?	Yes. Here, dtype=torch.float16 specifies the datatype used to store the model's weights when they are loaded into memory. Using float16 means each weight uses 16 bits instead of the usual 32 bits, reducing memory usage and often making inference faster.

	It means the chat formatted conversation is returned as a plain string, but not tokenized into IDs. Example: <pre>messages = [{"role": "user", "content": "What's the weather today?"}] # tokenize=False text = tokenizer.apply_chat_template(messages, tokenize=False) print(text) # e.g. "< user >\nWhat's the weather today?< end >\n< assistant >\n" # tokenize=True ids = tokenizer.apply_chat_template(messages, tokenize=True) print(ids)</pre>
apply_chat_template has tokenize= false. what is this?	# e.g. [128000, 128006, 882, ...]
does all the model supports < system >?	No. Every model family uses its own special tokens. Qwen uses < im_start >system, Llama 3 uses < start_header_id >system< end_header_id >, others differ again. That is why the notebook calls tokenizer.apply_chat_template. It looks up the correct format for that model, so you never write these tokens by hand.
what does return_tensors = "pt" mean	The type of tensors - pytorch tensors to be returned by the tokenizer.
Is it possible to show how to download models from hugging face and use it ??	The code snippets being shown here will automatically download the model. Else, you can also check in their website, for e.g. https://huggingface.co/Qwen/Qwen2.5-3B-Instruct for qwen3B instruction tuned model, you can check under files, you have the model files (as safetensors) which are the model parameters.

if the model is already downloaded and running in local, what is the max inference time? As it took almost 3-5 sec for the result?	There is no fixed maximum. Time equals tokens generated divided by tokens per second, so it scales with output length. For 1.5B on Colab GPU, roughly 20 to 50 tokens per second, so 80 tokens is a few seconds. That matches what you saw. On CPU it can be five to ten times slower. Cut max_new_tokens to cut the time.
When does one use Ollama? As with the approach demonstrated, user can run the model in local directly	Both approaches run the model locally, but Ollama simplifies the setup and serving process. Use Hugging Face Transformers when you need direct access to model internals, experimentation, or fine-tuning, use Ollama when you mainly want easy local inference through a REST/OpenAI-compatible API without managing PyTorch and Transformers yourself. Ollama is therefore convenient for running larger models such as 7B+ models as a local service while keeping the data private.
Does HF token is paid token?	The token itself is free. You create it from huggingface.co/settings/tokens with a free account. What is limited is usage. The free tier gives a small monthly inference credit and rate limits, so it is fine for learning and prototypes. Heavy or production use needs a paid plan. Downloading models to run locally stays free.
What are the other client we can use apart from OpenAI.	For OpenAI-compatible endpoints you rarely need another. The same OpenAI client works with HF router, Ollama, vLLM, Groq and Together, just change base_url.
What are the other client we can use apart from OpenAI.	Otherwise there are native SDKs, anthropic for Claude and google-genai for Gemini, plus framework clients like LangChain, LlamaIndex and LiteLLM if you want to switch providers from config.
How do we set any LLM like CLaude at enterprise level to scale it to 10000 + employees Amit Pandey_TA 10:38 AM For general use, take the enterprise plan and connect it to your identity provider with SSO and SCIM, plus data retention and admin controls. For apps your teams build, put an internal gateway in front of the provider. It holds the keys, enforces per-team budgets and rate limits, logs everything for audit, and lets you swap models without code changes. ...Thanks Sir ..I got the response..any good document to refer for	Start with the provider docs. Anthropic has enterprise and admin API documentation at docs.claude.com, covering SSO, SCIM, usage limits and audit logs. OpenAI has an equivalent enterprise admin section. For the gateway pattern, look at open-source LiteLLM Proxy and Portkey, their docs describe budgets, key management and logging well.

vllm manners means?	vLLM is the inference server we discussed, used for serving models efficiently on GPUs.
I get invalid username or password when I try to use the HF Token that was shared to me on my mail	That error usually means the token is being sent as a password rather than as a token. Check two things. Paste it exactly, no spaces or line breaks, and it should start with hf_. If you are using Colab userdata, confirm the secret name matches what the code reads, hf_key here, and that notebook access is toggled on.
when we learn how to host a model and what is infrastructure required to host a model like upgrad has hosted?	Depends on the model size you would want to host. Any form of GPU (like NVIDIA a40, a100,h100,h200,b200) that will be sufficient to hold the model will work.
For approach 1, when we say that we are downloading the model locally but using google colab then its basically not using our laptop cpu/gpu, its using cloud cpu/gpu?	Correct. Colab runs on a Google virtual machine, so the model downloads to that VM and uses its CPU or GPU, not your laptop. Your laptop is only the browser. It is still Approach 1 in the sense that the model runs inside your own Python process, but on borrowed hardware, and everything disappears when the session ends.
how do i host my own model like upgrad host	Using your own GPUs that will be sufficient to hold the model you want to host.
When I use the Upgrad token provided for a previous 30th August based colab, I got:: AuthenticationError: Error code: 401 - {'error': 'Invalid username or password.'}	Obtain an HF token from huggingface.co/settings/tokens and use it as YOUR_HF_TOKEN.
what is role of upgrade server/end point? only routing to open ai?	No, it does not touch OpenAI. UpGrad hosts an open model itself, Qwen2.5-7B-Instruct-AWQ, on their GPU server, most likely with vLLM. It only exposes an OpenAI-compatible API, which is why the same client works. Purpose is to give the class a shared GPU-backed model without everyone needing keys or hardware.
Follow up: I tried direct copy-pasting the access key, but no luck. I think some issue with the token that was shared to me.	Obtain an HF token from huggingface.co/settings/tokens and use it as YOUR_HF_TOKEN.
in upgrad approach, we need to use the qwen model only or we can try new model as well with the upgrad key?	Qwen model is hosted on upGrad server. You can definitely try new models available on Huggingface using your own HF_KEY.
whats the benefit of using open llm models from hugging face instead of using it directly from actual componay host as it already open? Will the same features not available on the main URL?	The benefit is one place, one API, one auth. Same transformers code works across every model, plus versioning, licences and community variants like quantised builds in the same repo.

in Zero Short Sentiment Classification why we are getting 5 sentiments per temp	These are not 5 sentiments. But the code will be executed 5 times for each temperature, to show the determinisitic nature.
in Zero Short Sentiment Classification why we are getting 5 sentiments per temp	It's like running for 5 seeds in Machine learnign setup.
if i run this calssiification task more tha once to check temp impact it says rate ;imit exceeded	I think that is the shared class endpoint throttling, not an error in your code. Everyone as of now is hitting it at once. Reduce the loop, run one or two temperatures instead of five, and add a short sleep between calls. If it persists, wait a minute and retry, or run that cell against the local model with ask instead.
Yes hit the rate limite error::----- ----- <pre> RateLimitError Traceback (most recent call last) /tmp/ipykernel_793/3074346448.py in <cell line: 0>() 6 print(f"\n=== temperature = {temp} ===") 7 for run in range(3): # run 3 times at each temp to show variance (or lack of it) ----> 8 print(f" [{run+1}] {ask_Qwen_7B(prompt, temperature=temp)}") 4 frames /usr/local/lib/python3.13/dist-packages/openai/_base_client.py in request(self, cast_to, options, stream, stream_cls) 1146 1147 log.debug("Re-raising status error") -> 1148 raise self._make_status_error_from_response(err.response) from None 1149 1150 break </pre> RateLimitError: Error code: 429 - {'detail': 'Rate limit exceeded (20/min)'}	
how does LLM host knowhow many tokens they can handle at a time? Is their any calculation using GPU? Also, from a user perspective, how can we know how many token this host can handle at any moment and not inducing slowness/error?	It's a GPU memory (VRAM) calculation — model weights + KV cache (which grows with context length × batch size × layers) must fit in available memory. Hosts calculate: remaining VRAM / memory per token. Techniques like quantization, continuous batching, and PagedAttention (vLLM) can help gain the number of tokens. From a user's view, we can't see live capacity directly. We only get the live signals e.g. errors.

Adding delay will fix the 20 Req per min issue	Yes. A 3 to 4 second sleep between calls keeps you under 20 per minute.
How do we know what temperature to choose, like is there any parameter or do we need to iterate and check for the best prompt	Start from the task, then verify. Cell 46 in your notebook has the table: 0 for classification and extraction, 0 to 0.3 for summarization, 0 to 0.2 for coding, 0.8 plus for creative work. Then confirm by running the same prompt three to five times at your chosen value. If you need consistency and answers differ, lower it. There is no parameter that picks it for you.
base_url="http://103.42.50.41:443/api1/", - How do I know this base_url?	The base URL specifies the address (endpoint) of the API server hosting or providing access to the LLM. In this case, UpGrad will provide the base URL.
<pre>print("temperature=0.0") print(ask_Qwen_7B(summary_prompt, max_tokens=100, temperature=0.0)) print("\ntemperature=0.8") print(ask_Qwen_7B(summary_prompt, max_tokens=100, temperature=2.0))</pre> temperature=2.0 pass why 2.0 it is?	Good catch, the label says 0.8 but the value passed is 2.0. I think it is a leftover from editing the cell. Not harmful, 2.0 just shows a stronger contrast against 0.0. Fix the label or change the value to 0.8 so they match.
can we say that Temperature 0 is same as greedy decoding?	Yes, effectively. At temperature 0 the model always takes the highest probability token, which is exactly greedy decoding.
So are these temperature calculated at run time based on the prompt of the user in real world LLM apps like Chatgpt, if it is a summarisation/rephrasing/classification etc?	Temperature is not calculated at run-time. It's a fixed or configured temperature used during inference. For classification, low temperature is preferred. For summarization, low-moderate temperature is good. For brainstorming or creative writing, high temperature will be good. A thing worth mentioning - in the real world LLM apps, temperature is not the only indicator, but many are factors like reasoning effort, model routing, system prompts, caching, internal inference policies (like when to call the web for some prompts) that play a role.
Hi . Please send this colab or let me know where this colab is I am not finding in today's resources.	https://colab.research.google.com/drive/1KIUMnstBxwW9wdl4m10lXr0detG-jnAl#scrollTo=TtkWRWZTWOQw
If we dont mention temperature at all, what is the default?	For the OpenAI-compatible APIs the default is 1.0.
If we dont mention temperature at all, what is the default?	ask_Qwen_7B defaults to 0, also deterministic.

	It's a concept used in vLLM https://arxiv.org/pdf/2309.06180
What is paged attention? has it been covered in any session?	On a high level, think of it as a virtual memory for an LLM's attention cache.
What is paged attention? has it been covered in any session?	PagedAttention manages the LLM's KV cache like an operating system manages RAM using pages, instead of requiring each request to occupy one large contiguous chunk of GPU memory.
What is paged attention? has it been covered in any session?	On an intuitive sense, think like if you don't use vLLM, it's like "reserve a huge parking space for every car, just in case it's a truck", if you use vLLM it's like "give each car small parking spaces as it actually needs them".
What is role of tokens and how to decide on number of tokens.	Tokens are the pieces the model reads and writes, roughly three quarters of a word in English. Everything is priced and limited in tokens. max_tokens caps the reply length only. Set it from the output you actually want, around 10 for a one-word label, 100 to 200 for a summary, more for code. Too low truncates mid-sentence, too high just wastes time and cost.
when we provide the max_token value to the model-> does this only factor for the final o/p text the model is trying to generate or does this also take into account the think tokens and other tokens the model may be utilizing right from ingestion of the i/p prompt till returning the final output ?	Input tokens are never counted in it. Your prompt is separate, billed as input and limited only by the context window. max_tokens caps generation. On a standard model that is just the visible reply. On a reasoning model the thinking tokens are generated too and do count against the budget, which is why a low cap can leave you with reasoning and no answer. Providers expose a separate reasoning or thinking budget for that reason.
is it using reasoning model in this ball example	No. Cells 41 and 42 call ask, which runs the local Qwen2.5-1.5B-Instruct, a standard model with no built-in reasoning. Cell 41 asks for the answer directly and it often gets 8. Cell 42 adds think step by step, and the same model gets 4. You are triggering chain of thought by prompt, not using a reasoning model.

In microsoft copilot enterprise we have 2 modes -work and web - web is used for external search in world wide web and work is used mostly for enterprise microsoft graph rag - its done to data protection of enterprise - is there some mode available where both web search and RAG of internal data both can be done in one same mode without loosing ur enterprise data to train the model OR some other tool model claude other gpt etc available commercially to do both . Or only option if you need to do this is to have locally open source llm where both internal , external are opened and u have it getting trained on both -thats the only possibility - or security tenant can be set in commercial llm available - request to pls comment - current state	One clarification first. Putting your data in a prompt is not training. Under enterprise agreements with Microsoft, Anthropic and OpenAI, prompts are excluded from training, so mixing web and internal data in one call does not leak anything into a model. Combined mode is available commercially. Claude Enterprise with web search plus connectors, ChatGPT Enterprise with web plus company knowledge. Copilot separates the two by design choice, not because the mix is unsafe. Self-hosting is not the only path. If you build it yourself, the app queries your vector store and a search API and passes both into the prompt, and that works with a hosted model too. Go local only if policy forbids data leaving your network.
is this math done by LLM is just next word prediction or does it perform any mathematical function?	It is only next token prediction. Reasoning models keep thinking and give explanations that. is also a part of next token prediction.
For Zero Shot, where are we saying - Think step by step	In cell 42 of the notebook, inside the problem string itself, "Think step by step and then give the answer." It is still zero-shot because you give no worked examples. Adding that phrase makes it zero-shot chain of thought. Cell 41 is the plain zero-shot version with "Give the answer directly."
why are we calling zero shot? since prompt is there	Shots mean examples. So no examples = zero shot.
Cant we give the examples to zero shot prompt?	It becomes few-shot in that case.
For section 1.3 Task 3 - question answer task - our company policy one why is there difference in outputs given by both ask() and ask_Qwen_7B	Two different models. ask runs the local Qwen2.5-1.5B, ask_Qwen_7B calls the hosted 7B on UpGrad's server, so the bigger model reasons better about the 30-day rule. Settings differ too. That cell passes temperature 2.0 to the 7B, which makes it erratic, while ask is called with no temperature, so it runs greedy. Set the 7B to 0 and the gap narrows.

can you repeate for prompts for ball?	Both use the same juggler problem, 16 balls, half are golf balls, half of those are blue. Cell 41 ends with "Give the answer directly." Small models often say 8, stopping after 16 divided by 2. Cell 42 ends with "Think step by step and then give the answer." The same model works through 16 to 8 to 4 and gets 4.
In the notebook /colab : context can be passed as a regular text , however in organiaztions there would be documents like ppt ,pdf , docx .Passing context will happen implicity when a chatbot is called correct ?	It won't be implicit, rather a text extraction pipeline from doc will be needed in between before sending the context. One needs to extract the text using parser or packages like python-docx to obtain the text. LLM has no awareness of PPT/PDF/DOCX. It can only comprehend the text in them.
What is the Endpoint (base url) that can be used in Approach 3: UpGrad hosted EndPoint? base_url="http://103.42.50.41:443/api1/", Is it some endpoints hosted by upgrad that we can use or its a locally hosted ?	it is supposed to be provided by UpGrad, please ask them for the credentials.
What is the Endpoint (base url) that can be used in Approach 3: UpGrad hosted EndPoint? base_url="http://103.42.50.41:443/api1/", Is it some endpoints hosted by upgrad that we can use or its a locally hosted ?	That's a privately hosted endpoint, set up by UpGrad for student access.
What is the Endpoint (base url) that can be used in Approach 3: UpGrad hosted EndPoint? base_url="http://103.42.50.41:443/api1/", Is it some endpoints hosted by upgrad that we can use or its a locally hosted ?	it is not loccaly hosted.

What is the Endpoint (base url) that can be used in Approach 3: UpGrad hosted EndPoint? base_url="http://103.42.50.41:443/api1/", Is it some endpoints hosted by upgrad that we can use or its a locally hosted ?	The IP address 103.42.50.41 is a private server they're providing where Qwen runs. Traditional Approach 3 runs Ollama on your own laptop at http://localhost:11434/. This UpGrad endpoint lets you use Qwen without needing hardware or installation, but data goes to their server. It's convenient for learning but differs from truly local Approach 3.
What is the Endpoint (base url) that can be used in Approach 3: UpGrad hosted EndPoint? base_url="http://103.42.50.41:443/api1/", Is it some endpoints hosted by upgrad that we can use or its a locally hosted ?	The IP address 103.42.50.41 is a private server they're providing where Qwen runs. Traditional Approach 3 runs Ollama on your own laptop at http://localhost:11434/. This UpGrad endpoint lets you use Qwen without needing hardware or installation, and data goes to their server.
where can I get the PDF DOC-20260704-WA0001..pdf?	download from here: https://drive.google.com/file/d/1ioKROv-ybbWmHPiamkQcwKQShr2NUWKh/view?usp=sharing
phrase question limit word	If it is prompt length, there is no word limit as such, only the model's context window in tokens. If it is about output, you control length with max_tokens or by saying it in the prompt, such as answer in under 30 words.
How this tool calling different from MCP?	Tool calling is the model side. It decides which function to call and produces the arguments, but your code must define the schema and run the function, exactly as in the notebook. MCP is the plumbing around that. It is a standard protocol so tools live on a server and any model or client can discover and use them without you rewriting schemas per app. Tool calling still happens underneath, MCP just standardises how tools are exposed and shared.
Difference between o shot and few shot prompting?	The difference is in number of in-context examples provided. Zero shot is when you don't give any examples but ask directly. Few shot is when you give some examples (like we have some samples in IQ tests before answerign the main question) and then ask the desired question.

can we have some more real world examples of tool calling	Fetching live data the model cannot know, <code>get_stock_price</code>, <code>get_order_status</code>, <code>check_inventory</code>, <code>get_weather</code>. Database queries, <code>run_sql</code> or <code>search_customer</code>, where the model turns a question into parameters and your code runs it safely. Actions, <code>create_ticket</code>, <code>send_email</code>, <code>book_appointment</code>, <code>refund_order</code>, always with a human confirmation step. Calculations, like the LCM and determinant examples, where exactness matters.
where is function definition here of <code>get_weather</code>	It is not on the slide, and that is the point. The tools array is only the description, name, purpose and argument schema, so the model knows what it can request. The real Python function lives in your code. See notebook cells 49 to 55, where <code>compute_determinant</code> is defined, the model returns a tool call, you run the function yourself, and pass the result back.
how does a model know that only specific tools to be called do we send the available tools along with input also	Yes. The tools list is sent with every request, so the model only ever sees the tools you passed in that call. <code>tool_choice</code> controls the behaviour, <code>auto</code> lets it decide or answer directly, or you can force a specific tool. Note the tool definitions count as input tokens each time.
If <code>tool_choice</code> is set to <code>auto</code> , the model decides whether to call the function . how should the application force a specific tool	Pass the tool name instead of <code>auto</code>. In the OpenAI format it is <code>tool_choice={"type": "function", "function": {"name": "get_weather"}}</code>, and the model must call that one. Other useful values are <code>"required"</code>, which forces some tool but lets the model pick, and <code>"none"</code>, which blocks all tools.
how does it decide to select the exact get weather tool ? does it uses tokens internally to figure out to use the tool ?	It matches your question against the tool name and description, which is why the description matters so much. Vague descriptions cause wrong tool choice. And yes, it is all tokens. The tool schemas are injected into the prompt as text, so they count as input tokens, and the tool call itself is generated as output tokens in a structured format.

There can be multiple predefined Tools? also what if the required parameter couldnt be extracted from the user prompt? What if user just mentioned -"How is the weather?"	tools is just a list, so you can register as many as you want. With tool choice = 'auto', model picks whichever tool fits the user's request. If a required parameter (say "required": ["location"]) is missing, the model may ask a clarifying question back to the user and no tool call is made. Sometimes, in poorly prompted cases and missing required parameters, the model may guess leading to hallucination.
in the prev step direct call, how do we know that LLM is calling this all calcuations and this much tokens wasted, is there any way to know that	Yes, two ways. The visible answer showed the whole factorisation working, which is the model doing arithmetic in text. For the count, every response carries a usage object. Print response.usage and you get prompt_tokens, completion_tokens and total_tokens. That is where the 425 on the slide comes from. Compare it against the tool version and the saving is measurable.
why cant we just train the model to return json schemas for tool based responses and we can just inut the required tool names and get the response in this way we can stop sending json schema in input everytime?	Yes sure that can be done. And some of the recent models are trained on how to use tools. But training is expensive right? So, if the models can be pushed to output in a fixed json schema (as they are good in following instructions), we can simply re-use in inference time, the existing models.
In tools array if the "description" of multiple tools is vague then model may not select the correct tool correct ?	Correct. Vague or overlapping descriptions are the main cause of wrong tool selection. Fix it by making each description say precisely what the tool does and when to use it, and by keeping the tools clearly distinct. If two overlap, mention the boundary explicitly, for example use this only for domestic orders.
if i give max tokens as 100, this is the limit of number of tokens on output or sum of tokens used for response and identifying the tool as well?	Output only. Your prompt and the tool schemas are input tokens and are not counted in it.
tool calling would also consume computation and tokenisation it may be less but it will still consume some token.. Right?	But the tool call itself is generated output, the function name and JSON arguments, so it does consume from that 100. Keep max_tokens large enough that the arguments are not truncated, otherwise you get broken JSON.
tool calling would also consume computation and tokenisation it may be less but it will still consume some token.. Right?	Yes you are right.
	When you use ChatGPT or other GPT models, you can check in the cost interface, tool calls cost money as well. So yes, tokens get used there.

how does the LLM know about the compute LCM function	From the tools list you pass in the call. That schema, the name <code>compute_lcm</code>, the description, and the <code>a</code> and <code>b</code> parameters, is inserted into the prompt as text, so the model reads it like any other instruction. It never sees your Python code. It only knows a tool with that name exists and what arguments it takes.
how LLM knows that now i need Tool and can not solve on its own?	That's a very good question! Currently, no one knows when do the models need to use the tool. perhaps the models themselves do not know. At times, the models call tools if there are questions outside their knowledge cutoff date. The system prompts for the models perhaps determine when to call tools. Check https://github.com/asgeirtj/system_prompts_leaks/blob/main/OpenAI/gpt-5-thinking.md#:~:text=medium%20or%20long-,Decision%20boundary,-If%20the%20user for some instructions when to call the web tool.
how LLM knows that now i need Tool and can not solve on its own?	But it is not guaranteed that the models calls tools when it is actually needed,. They also make unnecessary calls. Some of the research I have been involved in shows that toolcalls are made unnecessarily leading to costs, but no tool (<code>tool_choice = 'none'</code>) leads to the same or better answer. You may check one of our works here - https://arxiv.org/pdf/2605.00737
Trying to understand if i have a defined deterministic task why would i need LLM? Or does it integrate specific steps using tool calling inside an overall AI workflow? I understand here we are using toy examples so the application is not very clear to me	Fair point, and you have answered it yourself in the second sentence. The deterministic step does not need an LLM. The model is there for the messy part around it. The user types free text, "did my order from last Tuesday ship yet". The model turns that into the right function and the right arguments, decides which of twenty tools applies, chains several if needed, then turns the raw JSON result back into a sentence. Your code still does the actual work, deterministically. So the LLM handles interpretation and orchestration at the edges. If your input is already structured and the path is fixed, you are right, just write the code.
in current context -- is tool function develop by developer?	Yes. You write both the schema and the actual function, as with <code>compute_lcm</code> and <code>compute_determinant</code> in the notebook.
What are some real world scenarios where in we should be using Tool calling. I can think of Company level chatbots... like Swiggy bot or any other bot having a predefined set of options. Can you please help me with some more options?	If you want to look for current information, you need web access leading to need for web tool calling. If you want to edit some photos (very much used in chatgpt or other platforms), tools like adobe get called for the purpose.

What are some real world scenarios where in we should be using Tool calling. I can think of Company level chatbots... like Swiggy bot or any other bot having a predefined set of options. Can you please help me with some more options?	If one needs to fact check, web search against a live authentic source will be needed,
if the tool executes on the local machine. it can allow give local context to LLM. Ilike local files or settings?	Yes, that is exactly how local agents work. You write a tool like read_file or list_directory, the model asks for it by name, your code reads the file and returns the content, which then enters the prompt as context.
In form extraction, is this OCR - meaning are we saying LLM can interpret images and summarize fields into JSON	Not exactly. OCR specifically means extracting text from an image/document. In form extraction, a multimodal LLM can go further: it can read the form, understand the fields and their values, and directly produce structured JSON.
this colab link i now working for me. can you please reshare	https://colab.research.google.com/drive/1KIUMnstBxwW9wdl4ml0lxr0detG-jnAl#scrollTo=TtkWRWZTWOQw
i pydantic only compatible with OpenAI or any interface/models	Pydantic is just a Python validation library, nothing to do with OpenAI. It works with any model output, any provider, even plain API responses.
If we define the exact response json structure in the prompt(with examples) with all valid dtypes, will the advanced llms like claude opus or gpt4 return invalid jsons	Yes, they can still return invalid JSON if you only specify the structure in the prompt. Even advanced models can occasionally add extra text, omit fields, use the wrong type, or produce malformed JSON.
A couple of days back I fed a screen shot of a device console capturing some problem condition and runtime logs which I wanted to investigate. I fed this to a tool that uses gemini AI and asked it to extract the text from the image for processing it with claude cli. This tool generated some random text unrelated to the image which kind of puzzled me. I provided the exact same prompt to Opus and GPT 5.6 and they were able to generate a plain text version of the text in the screen shot picture. What does it say about the model behaviour? I am just trying to correlate it with the topic being discussed currently.	The wrapper tool may not have sent the image at all, or downsized it badly, or routed to a small vision model. Random unrelated text usually means the model got a blank or unreadable input and generated plausible-looking filler rather than saying it saw nothing. The behaviour itself is the failure mode from these slides. The output was well formed and confidently wrong, which is exactly the parse-rate versus accuracy gap on the evaluation slide. Structure and formatting never guarantee correctness. For OCR of console logs, the practical fix is a verification step, ask the model to return only text it can actually read and mark unclear regions, then eyeball it against the screenshot before feeding it downstream.
I assume Pydantic only helps with the validation of the structure of the output as per the defined schema. However, the result could still be incorrect or incomplete, right?	Exactly. Pydantic guarantees shape and types, not truth. A perfectly valid Invoice object can carry the wrong vendor or a hallucinated total.
what is the relevance of tool_choice here?	Relevance of tool_choice is whether you are giving the model the agency to decide by itself to call the tool (auto), or forcing them to use tools or not to use tool (none).

Does Form Data extraction usecase in earlier slide - means that we can use form images as an input to LLM to summarize it into JSON ?	Yes, with a vision-capable model you can send the image directly and get structured JSON back. The notebook takes the other route, PDFReader extracts the text first, then the model structures it. Both work. Use text extraction when the PDF has a text layer, and vision when it is a scan or the layout matters.
Is tool calling a extra overhead in terms of tokens usage?	Tool calls do cause an extra overhead in terms of token usage - the entire tools schema (name, description etc.) is sent along with input tokens. Model's tool call output tokens also add in to the tokens. At times, the tool call output tokens go back as input to the model and used in iterations - so there tokens get used up as well.
who maintains pydantic library	https://pydantic.dev/ - it's maintained by pydantic company.
Whats this pydantic terminology I didn't get the context	Pydantic is just a Python library. You define a class listing the fields you expect and their types, and it checks any incoming data against that, converting where it can and raising an error where it cannot. Here it is used for LLM output. You declare what the JSON should look like, Pydantic generates the schema to send to the model, then validates what comes back so your code gets a proper typed object instead of a risky string.
@Amit followup question - Vision LLM is hallucinating sometimes e.g in checkbox states determination. Is there a way to handle such hallucinations. Thanks for prompt replies	Coming strictly from my experience - Checkboxes are a known weak spot for vision models. A few things that help. Crop and zoom to the checkbox region instead of sending the whole page, and increase image resolution. Force a constrained output, an Enum of checked, unchecked, unclear, so the model cannot waffle. Ask for one field at a time rather than the whole form in one pass. Also let it say unclear and route those to a human. And for pure checkbox detection, a small CV model or template matching is often more reliable than an LLM.
@Amit followup question - Vision LLM is hallucinating sometimes e.g in checkbox states determination. Is there a way to handle such hallucinations. Thanks for prompt replies	I have tried yolov10 DocLayout to identify section, and send it to the LLM as parts instead of whole document. But there could be other approaches.

Instructor works with Langchain or langchain is a type of instructor itself ? what is the difference b/w instructor and Orchestrator?	Instructor is mainly a structured output/LLM interface library - its goal is to reliably produce structured, validated output. Langchain is broader - it provided abstractions for building LLM applications - it's a framework for connecting LLMs with tools, data and applications. Langchain and Instructor can be used alongside, they are not alternatives. Orchestrator is the component that decides what happens next. Think of a question related to billing. Orchestrator decided whether to call the billing API or not. AI application - restaurany LLM is the Chef. Instructor makes sure the chef's dish follows a specific recipe/format. LangChain is the kitchen framework connecting ingredients, tools and processes. Orchestrator is the head chef/manager deciding what should happen next.
If the LLM does not respond according to the pydantic schema, does the pydantic take care of the fallback and go back to LLM to fetch the response in the required format?	Pydantic itself does not. It only raises a ValidationError, nothing more. The retry comes from the wrapper. Instructor, in the notebook, catches that error, sends the message back to the model, and retries a few times automatically.
thanks @Amit - I am working on these types of variation on https://cioms.ch/wp-content/uploads/2017/05/cioms-form1.pdf . Will check Yolo.	Sure.
thanks @Amit - I am working on these types of variation on https://cioms.ch/wp-content/uploads/2017/05/cioms-form1.pdf . Will check Yolo.	Had a quick look at the pdf. The CIOMS form is a fixed layout, which works in your favour in my opinion. Since the field positions are fixed, template matching or a layout model gives you reliable crop regions, and you only run the vision model on each small region. Am positive it should work.
i think today sessions are dragged so much	The recording will cover, so nothing is lost. Pick one thread you care about, maybe structured output for your own use case. Rest you can reach out to the academic team for further queries or suggestions.

So if a tool description is not accurate the LLM can select a wrong tool?	Yes, that is the main failure mode. The model has only the name and description to go on, so a vague or overlapping description leads it to the wrong function or to skip tools entirely.
via tools, we are sending more context to llm?	Yes, in two ways. The tool schemas themselves are added to the prompt as input tokens on every call, so ten tools cost more than two. Then the tool result goes back as another message, which is more context again. That is the trade-off, tool calling costs extra tokens but buys accuracy and access to live data.
LLM can also hallucinate in calling the wrong tool. How should we ensure that LLM calls the right tool everytime ?	You can specify which tool to call thus confining it. It is difficult to ensure LLM calls the right tool without giving it the tool to call for. Mostly, it calls the right tool depending on the question. The hallucination part kicks in when the tools fetch answers from different places (say websites), and then the model has to synthesize answer to return back to the user. At that time of synthesis, it is a real challenge for the models to understand the veracity of the information and then compile the answer.
Any guidelines on how to write the tool or enhance the tool	Some suggestions- Keep each tool doing one clear thing, and name it like a verb, <code>get_order_status</code>, not <code>order_handler</code>. Write the description for the model, not for a developer. Say what it does and when to use it, and mark the boundary if another tool is close. Do the same for every parameter, since that is how the model knows what to fill in. Use strict types, Enums for fixed choices, and mark required fields. Validate inputs in your code anyway, never trust the arguments blindly. Return small, clean results rather than a full API dump, and return a readable error message on failure so the model can recover or ask the user.
Can someone share the PDF that is used in the lesson?	The invoice PDF is a sample the instructor uploaded to Colab, so it is not in the shared notebook. Let me flag it to the team to share along with the materials.

can you please share KB/wenb link to install and get hands on on ollama	Start at ollama.com/download for the installer, Mac, Windows and Linux all covered. The model list is at ollama.com/library. For hands on, github.com/ollama/ollama has the README with commands and the API reference. After installing, <code>ollama pull llama3.2</code> then <code>ollama run llama3.2</code> gets you going in two minutes.
how does production agents like Claude understand whether to call this pdf reader or some other extarctor which extracts info from image do they use CNN's for identifying the image or attachment and call respective tool?	No CNN involved for that decision. It is much simpler, the routing is done on metadata the model can read as text, the file extension and MIME type. If it is a .pdf, the PDF tool description matches, if it is a .png, the image tool does. Often it is not even the model deciding. The application layer inspects the file type in code and picks the extractor, then only the extracted text goes to the model. Where vision models do get used is after that, reading the content of the image, not choosing which tool handles it.
team i am getting the file not found error when trying to read the pdf from the below directory: Errno 2] No such file or directory: '/content/DOC-20260704-WA0001.pdf'	it's mentiond in the colab note book in cell C ("the doc :") and present here - https://drive.google.com/file/d/1ioKROv-ybbWmHPiamkQcwKQShr2NUWKh/view
team i am getting the file not found error when trying to read the pdf from the below directory: Errno 2] No such file or directory: '/content/DOC-20260704-WA0001.pdf'	Cell C Pydantic Class Creation
how to train our LLM to call the right tool	There are ways to call the right tools during training. You can check here for Qwen - https://arxiv.org/pdf/2505.09388